Process Synchronization Contd2

Dr. Jit Mukherjee

The Reader-Writers Problem

- Two type of processes
  - Read the database or files
  - Write into them
- Two readers -> no problem
- One reader and one Writer
- Multiple Writers
- No reader should wait for other readers.
- If writer is waiting to access, no new reader start reading.
- Both lead to starvation
- mutex ->wrt -> 1,readcount ->0

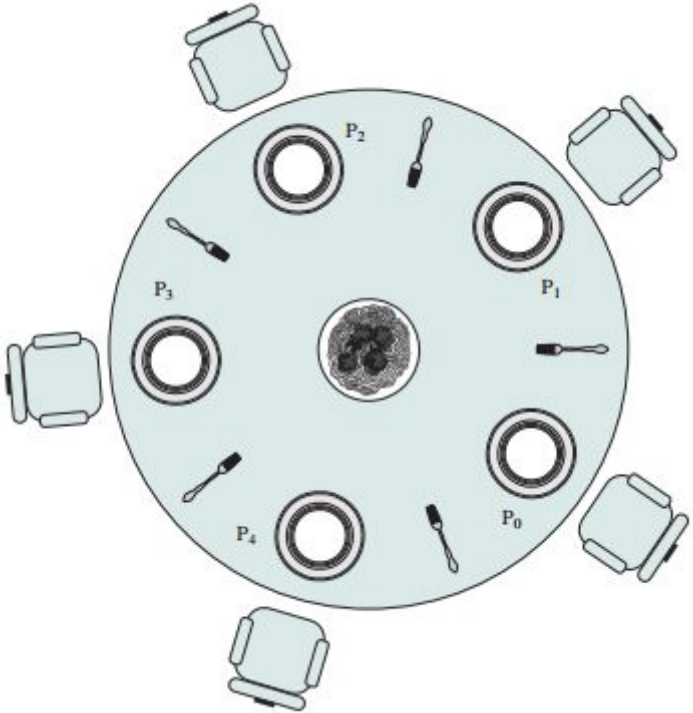
```
semaphore mutex, wrt;
int readcount;
```

```
do {
        wait (wrt);
        // writing is performed
        signal (wrt);
}while (TRUE);
```

```
do{
        wait(mutex);
        readcount++;
        If (readcount == 1)
                wait(wrt);
        signal(mutex);
        // reading is performed
        wait (mutex);
        readcount - -;
        if(readcount == 0)
                signal(wrt);
        signal (mutex);
}while (TRUE);
```

The Dining-Philosophers Problem

- Five philosophers -> think and eat. Circular table
  - Five chairs
  - Five chopsticks
- When one philosopher think does not talk
- Pick chopsticks closes to them
  - One chopstick at a time
  - Free chopsticks
  - Eats when two chopsticks in hand
- Finishes eating -> put both down.
- Several resource between several process
- Each chopsticks with a semaphore -> 1
  - All hungry, picks left chopsticks



The Dining-Philosophers Problem

1. Allow four philosophers to be sitting simultaneously in the table
2. Allow philosopher to pick up chopsticks only if both are available
3. Asymmetric solution
   a. Odd picks left first then right
   b. Even picks right first then left

```
Semaphore chopsticks [5];


do {
        wait (chopsticks[i]);
        wait (chopsticks[(i+1)%5]);
        .......
        //eat
        .......
        signal (chopsticks[i]);
        signal (chopsticks[(i+1)%5]);
        .......
        // think
        .......
}while (TRUE);
```

Semaphore timeouts

- User task is stuck until that semaphore is unlocked. While it is stuck waiting for the semaphore, it keeps track of how long it has been waiting
- If it is stuck for more than 30 seconds, this is considered a semaphore timeout and in debug mode a message will be logged to the console.
  - The task will continue to wait for the semaphore, timing out every 30 seconds, until the semaphore is unlocked or the task is ended.
- Reasons for semaphore timeouts
  - A heavy load on the server is causing processes to be delayed from releasing semaphores.
  - A process has crashed while holding a semaphore, causing other processes to block when trying to acquire the semaphore.
  - Two tasks are waiting on each other and neither task is able to break the loop. In the simplest case, thread A is trying to get a semaphore which is owned by B, while B is trying to get a different semaphore which is owned by A. More complex combinations are also possible: A wants a semaphore owned by B, who wants a semaphore owned by C, who wants a semaphore owned by A, etc.
  - If a process was to fail to set a semaphore during execution, another process dependent on the semaphore will be blocked awaiting the semaphore.

Monitors

- Using semaphore incorrectly creates race conditions
  - Even if single process is not well behaved
- A process interchanges the order of wait () and signal () -> signal (mutex); //critical section// wait(mutex);
  - Several processes will be executing it's critical section simultaneously
- If a process replaces a signal (mutex) with wait (mutex) -> wait(mutex); //critical section// wait(mutex);
  - A deadlock will occur
- If a process omit wait () or signal ()
  - Either mutual exclusion is violated or deadlock will occur
- To deal with such error - one high level synchronization construct -> Monitor
- Provided by - Concurrent Pascal, Modula, Modula-2, Mesa, and Java.

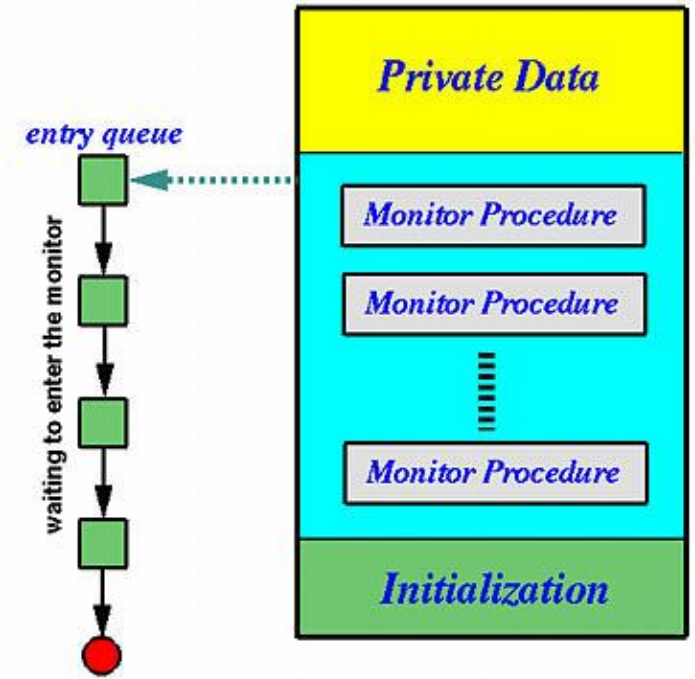
Monitor Contd.

- Monitors are abstract data types and contain shared data variables and procedures.
- It is the collection of condition variables and procedures combined together in a special kind of module or a package.
- The shared data variables cannot be directly accessed by a process.
- The processes running outside the monitor can't access the internal variable of the monitor but can call procedures of the monitor.
- Only one process can be active in a monitor at a time.
- Other processes that need to access the shared variables in a monitor have to line up in a queue and are only provided access when the previous process release the shared variables.

```
monitor monitorName
{
    data variables;

    Procedure P1(....)
    {

    }

    Procedure P2(....)
    {

    }

    Procedure Pn(....)
    {

    }

    Initialization Code(....)
    {

    }
}
```

Components of Monitors

- Initialization: - Initialization comprises the code, and when the monitors are created, we use this code exactly once.
- Private Data: - outside the monitor, private data is not visible.
- Monitor Procedure: - Monitors Procedures are those procedures that can be called from outside the monitor.
- Monitor Entry Queue: - contains all threads that called monitor procedures but have not been granted permissions
- Condition variables are synchronization primitives that enable threads to wait until a particular condition occurs.
- a monitor can be considered a mini-OS with limited services.



Monitor Contd.

- Only operations invoked on condition variables Wait () and Signal ()
  - Wait () - immediately suspended and put into the waiting queue of that condition variable
  - To indicate a particular event occurs, a thread calls the signal method on the corresponding condition variable
  - Signal and wait / Signal and continue
- In semaphores, Wait() does not always block the caller
  - In condition variables Wait() always blocks the caller.
- In semaphores Signal() either releases a blocked thread, if there is one, or increases the semaphore counter.
  - In condition variables Signal() either releases a blocked thread, if there is one, or the signal is lost as if it never happens.
- Monitors have the advantage of making parallel programming easier and less error prone
- Monitors have to be implemented as part of the programming language . The compiler must generate code for them. This gives the compiler the additional burden
  - absence of concurrency is the major weakness in monitors and this leads to weakening of encapsulation

Dining-Philosophers - Monitor

- Restriction - a philosopher may pick up chopsticks only if both available
- Each philosopher before starting to eat mush invoke pickup()
  - Suspension of the process
- Completion putdown()
- dp.pickup(i); //eat// dp.putdown(i);

```
monitor dp {
        enum {thinking, hungry, eating} state [5];
        condition self [5];
        void pickup (int i) {
                state [i] = hungry;
                test(i);
                If (state[i] != eating)
                        self.wait();
        }
        void putdown (int i){
                state[i] = thinking
                test ((i+4)%5);
                test ((i+1)%5);
        }
        void test (int i){
                if((state[(i+4)%5]!= eating) &&
(state[i]==hungry) && (state[(i+1)%5]!=eating)){
                        state[i] = eating;
                        self[i].signal();
                }
        }
        initialization(){
                for(int i=0;i<5;i++)
                        state[i] = thinking;
        }
}
```